

3. 가상 스레드 생성하고 실행하기

가상 스레드를 만드는 방법은 크게 세 가지다. 빌더로 만드는 `Thread.ofVirtual()`, 권장 방식인 `Executors.newVirtualThreadPerTaskExecutor()`, 그리고 한 줄짜리 편의 메서드 `Thread.startVirtualThread()`.

1. Thread.ofVirtual() — 빌더로 만들기

공식 문서에 따르면,

- `Thread.ofVirtual()` 메서드를 호출해 가상 스레드 생성을 위한 `Thread.Builder` 인스턴스를 만든다.
- `ofVirtual()` 은 가상 스레드, 또는 가상 스레드를 만드는 `ThreadFactory` 를 생성하기 위한 빌더를 반환한다.

```
Thread thread = Thread.ofVirtual().start(
    () -> System.out.println("Hello World")
);
thread.join();
```

`Thread.Builder` 를 쓰면 이름 같은 공통 속성을 설정할 수 있다. 참고로 `Thread.Builder.OfPlatform` 하위 인터페이스는 플랫폼 스레드를, `Thread.Builder.OfVirtual` 은 가상 스레드를 만든다.

```
Thread.Builder builder = Thread.ofVirtual().name("MyThread");
Runnable task = () -> {
    System.out.println("Running thread");
};
Thread t = builder.start(task);
System.out.println("Thread t name: " + t.getName());
t.join();
```

빌더는 재사용해서 여러 스레드를 만들 수도 있다. 문서 예제처럼 `name("worker-", 0)` 같이 접두사와 시작 번호를 주면, `worker-0`, `worker-1` 처럼 자동으로 번호가 매겨진다.

```
Thread.Builder builder = Thread.ofVirtual().name("worker-", 0);
Runnable task = () -> {
    System.out.println("Thread ID: " + Thread.currentThread().threadId());
};
Thread t1 = builder.start(task);    // name "worker-0"
t1.join();
Thread t2 = builder.start(task);    // name "worker-1"
t2.join();
```

2. Executors.newVirtualThreadPerTaskExecutor() — 권장 방식

작업 관리를 애플리케이션의 나머지 코드에서 분리하고 싶을 때 쓰는 방식이다. 이 executor 는 `ExecutorService.submit(...)` 이 호출될 때마다 새 가상 스레드를 만들어 작업을 실행하고, `Future` 를 반환한다.

```
try (ExecutorService myExecutor =
    Executors.newVirtualThreadPerTaskExecutor()) {
    Future<?> future = myExecutor.submit(() → System.out.println("Running
thread"));
    future.get();                // 작업 완료를 기다림 (블로킹)
    System.out.println("Task completed");
}
// try-with-resources 사용
```

여기서 주목할 점은 `try-with-resources` 다. 공식 채택 가이드는 다음을 설명한다.

- `ExecutorService` 자체가 가볍고, 일반 객체처럼 새로 만들 수 있다.
- `try` 블록이 끝날 때 암묵적으로 호출되는 `close()` 가, 제출된 모든 작업(즉 executor가 생성한 모든 가상 스레드)이 종료될 때까지 자동으로 기다린다.

그래서 별도의 `shutdown()` / `awaitTermination()` 없이도 깔끔하게 정리된다.

다음은 여러 서비스를 동시에 호출하는 **fan-out 시나리오**에 적용한 공식 문서 예제다.

```
void handle(Request request, Response response) {
    var url1 = ...
    var url2 = ...
    try (var executor = Executors.newVirtualThreadPerTaskExecutor()) {
        var future1 = executor.submit(() → fetchURL(url1));
        var future2 = executor.submit(() → fetchURL(url2));
        response.send(future1.get() + future2.get());
    } catch (ExecutionException | InterruptedException e) {
        response.fail(e);
    }
}
```

공식 문서는 이처럼 **수명이 짧은 동시 작업에도 가상 스레드 생성을 권장**한다.

3. Thread.startVirtualThread() — 한 줄로 만들기

가상 스레드를 만들고 바로 시작하는 정적 편의 메서드이다. 공식 문서는 "가상 스레드를 생성해 작업을 실행하고, 실행되도록 스케줄한다"고 설명한다.

```
Thread t = Thread.startVirtualThread(() → System.out.println("Hello"));
```

```
t.join();
```

`Thread.ofVirtual().start(task)` 와 사실상 같은 일을 하지만, 빌더 설정(이름 등)이 필요 없을 때 가장 간단하다.